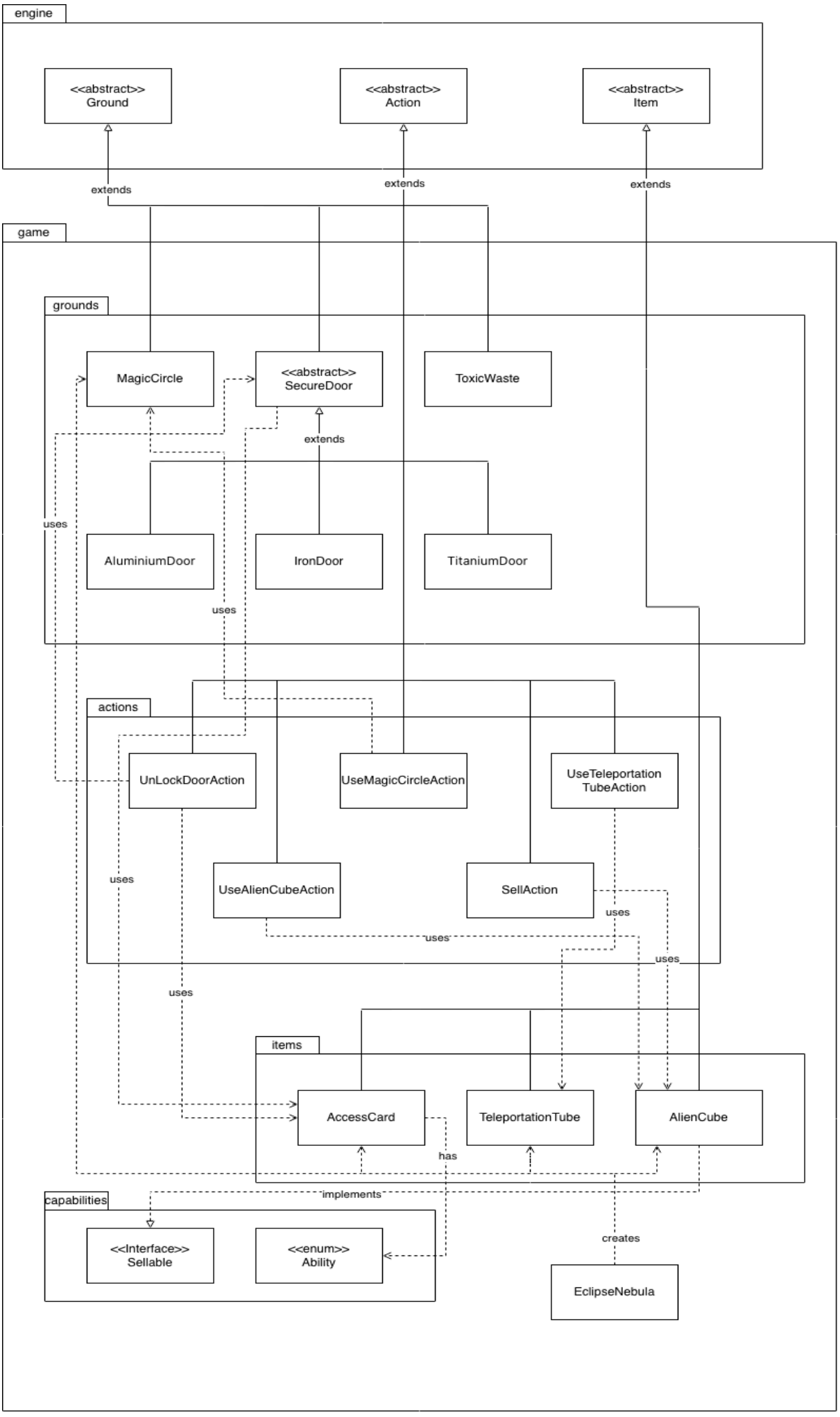
Design 1:Abstract class SecuredDoor with three subclasses
Each door type (AluminiumDoor, IronDoor, TitaniumDoor) extends an abstract base class SecuredDoor. The common logic (clearance checking, unlock action) is in SecuredDoor, and each subclass only overrides requiredClearance() and unlockEffect().

Pros	Cons
Following the DRY principle, the common door logic is centralized in `SecuredDoor` and is not repeated across the three classes.	It requires additional abstract classes, which increases the complexity of the class structure.
Following the OCP principle, adding a new door type requires only creating a new subclass; the existing code does not need to be changed.	
Following the SRP principle, each class defines only its own unique unlocking effects.	
Following the DIP principle, `UnlockDoorAction` depends on the abstract class `SecuredDoor` rather than a specific door class.	

Design 2:Three independent door types
Each door type directly inherits from `Ground` and implements its own access card verification and unlocking logic.

Pros	Cons
The structure is simpler and does not require abstract classes.	This violates the DRY principle, as the access card verification logic is repeated across three classes.
Each class is completely independent, making it easier to understand when read on its own.	This violates the OCP principle,adding new door types requires copying and changing existing door code.
	This violates the DIP principle; `UnlockDoorAction` needs to depend on each specific door type individually.


engine
<<abstract>>
Ground
<<abstract>>
Action
<<abstract>>
Item
extends
extends
extends
game
grounds
MagicCircle
<<abstract>>
SecureDoor
ToxicWaste
extends
uses
AluminiumDoor
IronDoor
TitaniumDoor
uses
actions
UnLockDoorAction
UseMagicCircleAction
UseTeleportation
TubeAction
uses
UseAlienCubeAction
SellAction
uses
uses
uses
uses
items
AccessCard
TeleportationTube
AlienCube
has
implements
capabilities
<<Interface>>
Sellable
<<enum>>
Ability
creates
EclipseNebula

I chose design 1. Although it introduces an additional abstract class, the benefits of centralizing the common logic within `SecuredDoor` significantly outweigh the added complexity. This design follows the DRY, OCP, SRP, and DIP principles, making the code easier to maintain and extend. If a new door type needs to be added in the future, all that's required is to create a new subclass of `SecuredDoor` ; no changes to existing code are necessary.